

1. CasinoShiz: математическая модель казино-бота

1.1. 1. Обозначения

- B_t - баланс игрока перед операцией.
- s - ставка игрока.
- p - выплата игроку после исхода.
- m - множитель выплаты, так что $p = s * m$.
- $RTP = E[p] / s$ - return-to-player без внешних бонусов.
- $edge = 1 - RTP$ - ожидаемая доля ставки, остающаяся системе.
- $net = p - s$ - чистый результат игрока.

В большинстве игр:

$$B_{after} = B_{before} - s + p$$
$$E[net] = E[p] - s = s * (RTP - 1)$$

Операции списания и начисления выполняются идемпотентно через operation id.

1.2. 2. Базовая экономика монет

1.2.1. 2.1. Ставки и выплаты

Для игр с предварительной ставкой:

```
debit(s, reason = "<game>.bet")
roll outcome
credit(p, reason = "<game>.payout") if p > 0
```

Ставка ограничивается MaxBet в настройках конкретной игры.

1.2.2. 2.2. Переводы

Transfer отправляет получателю n монет, а с отправителя списывает:

```
fee_raw = n * FeePercent
fee_half = round_away_from_zero(fee_raw * 2) / 2
fee = max(MinFeeCoins, round_away_from_zero(fee_half))
total_debit = n + fee
```

Дефолт: FeePercent = 0.03, MinFeeCoins = 1. Комиссия перевода является sink монет.

1.2.3. 2.3. Daily bonus

$bonus = \min(MaxBonus, \text{floor}(balance * \text{PercentOfBalance} / 100))$

Дефолт: PercentOfBalance = 0.35, MaxBonus = 8, MaxCatchUpDays = 14. Это faucet монет, ограниченный сверху 8 монетами за локальный день.

1.2.4. 2.4. Gas tax и bank tax

```
GasDefault = 0.0285
GasModifier = sqrt(2)
GasFunction(x) = max(1, ((x + 1) ^ log10(x + 1) - 1) / 39.15)
```

```
if x < 10:
    gas(x) = round(GasFunction(x) * GasModifier)
else:
    gas(x) = round(x * GasDefault * GasModifier)
```

Текущее поведение: $gas(9) = 1$, но $gas(10) = 0$ из-за округления во второй ветке.

```
if bank < 70: bank_tax = -2
else if bank < 120: bank_tax = 0
else: bank_tax = round(max(4, min(gas(floor(bank)), 25)) / 2)
```

1.3. 3. Модель отдельных игр

1.3.1. 3.1. Telegram dice games

Исходы Telegram dice считаются равномерными.

Модуль	Исходы	Множители выигрыша	RTP
dicecube	1..6	4 -> x1, 5 -> x2, 6 -> x2	5/6 = 83.3333%

darts	1..6	4 -> x1, 5 -> x2, 6 -> x2	5/6 = 83.3333%
bowling	1..6	4 -> x1, 5 -> x2, 6 -> x2	5/6 = 83.3333%
football	1..5	4 -> x2, 5 -> x2	4/5 = 80%
basketball	1..5	4 -> x2, 5 -> x2	4/5 = 80%

Для dicecube множители Mult4, Mult5, Mult6 конфигурируемые:

$$RTP = (Mult4 + Mult5 + Mult6) / 6$$

При дефолте 1,2,2: RTP = 83.3333%, edge = 16.6667%.

1.3.2. 3.2. Slots (/dice, Telegram slot machine)

Текущий DiceOptions.Cost = 9. Сверху добавляется gas:

$$loss = Cost + gas(Cost)$$

Для дефолта gas(9) = 1, значит loss = 10.

Telegram slot value декодируется в 3 барабана по 4 символа:

```
roll_0 = ((value - 1) >> 0) & 3
roll_1 = ((value - 1) >> 2) & 3
roll_2 = ((value - 1) >> 4) & 3
```

index	symbol	stake price
0	bar	1
1	cherry	1
2	lemon	2
3	seven	3

Выплата:

$$rolls_sum = sum(stake_price[roll_i])$$

```
three seven -> 77
three lemon -> 30
three cherry -> 23
three bar -> 21
two seven -> 10 + rolls_sum
two lemon -> 6 + rolls_sum
any other pair -> 4 + rolls_sum
no pair -> rolls_sum - 3
```

По всем 64 равновероятным исходам:

$$sum(payouts) = 610$$

$$E[p] = 610 / 64 = 9.53125$$

Для дефолтного loss = 10:

$$RTP = 9.53125 / 10 = 95.3125\%$$

$$edge = 4.6875\%$$

$$E[net] = -0.46875 \text{ монеты за spin}$$

1.3.3. 3.3. Blackjack

Правила settlement:

```
if player_total > 21: payout = 0
else if player_blackjack and not dealer_blackjack: payout = bet + floor(3 * bet / 2)
else if player_blackjack and dealer_blackjack: payout = bet
else if dealer_total > 21: payout = 2 * bet
else if player_total > dealer_total: payout = 2 * bet
else if player_total < dealer_total: payout = 0
else: payout = bet
```

Natural blackjack платит 2.5x ставки с целочисленным $\text{floor}(1.5 * \text{bet})$ в бонусной части. Замкнутого RTP в коде нет: он зависит от стратегии игрока hit/stand/double.

1.3.4. 3.4. Pick

Пользователь задает N вариантов и выбирает k backed variants.

```
P(win) = k / N
gross = s * N / k
base_payout = floor(gross * (1 - HouseEdge))
```

Дефолт: HouseEdge = 0.05, MinVariants = 2, MaxVariants = 10, MaxBet = 5000.

Без streak bonus:

```
E[p] ~ s * (1 - HouseEdge)
RTP ~ 95%
edge ~ 5%
```

Streak bonus:

```
factor = min(max(0, streak_after - 1), StreakCap)
bonus = floor(s * factor * StreakBonusPerWin)
total_credit = base_payout + bonus
```

Дефолт: StreakBonusPerWin = 0.5, StreakCap = 4, ChainMaxDepth = 5. При длинной серии побед streak bonus может перекрывать house edge.

1.3.5. 3.5. Pick lottery

```
pot = sum(stake_i)
if entrants < MinEntrantsToSettle:
    refund all stakes
else:
    winner ~ Uniform(entries)
    fee = floor(pot * HouseFeePercent)
    payout = pot - fee
```

Дефолт: MinEntrantsToSettle = 2, HouseFeePercent = 0.05. При равных ставках RTP около 95%.

1.3.6. 3.6. Horse race

Для лошади h:

```
S_h = total stake on horse h
P = total pot
```

```
if S_h = 0:
    k_h = 1.0
else:
    k_h = floor(((P - S_h) / (1.1 * S_h)) * 1000) / 1000 + 1
```

Если h победила:

```
payout_i = floor(stake_i * k_h)
total_payout_h ~ S_h + (P - S_h) / 1.1
house_hold_h ~ (P - S_h) / 11
```

Hold зависит от распределения ставок и победителя. Для полного EV нужно отдельно проверить распределение SpeedGenerator.GenPlaces(HorseCount).

1.4. 4. Meta progression

1.4.1. 4.1. XP за игру

Формула читается из meta_seasons.config.xp. Если JSON отсутствует или битый, используются дефолты:

```
play = 5
win = 25
loss = 2
stakeMultiplier = 0.01
minXpPerGame = 1
maxXpPerGame = 500
```

Расчет:

```
base_xp = is_win ? config.xp.win : config.xp.loss
stake_xp = floor(max(0, stake) * config.xp.stakeMultiplier)
xp_delta = clamp(config.xp.play + base_xp + stake_xp, minXpPerGame, maxXpPerGame)
```

Для дефолта:

```
win: xp = clamp(30 + floor(0.01 * stake), 1, 500)
loss: xp = clamp(7 + floor(0.01 * stake), 1, 500)
```

1.4.2. 4.2. Rating

Rating читается из meta_seasons.config.rating:

```
enabled = true
start = 1000
winDelta = 16
lossDelta = -12
```

Расчет:

```
rating_delta = enabled ? (is_win ? winDelta : lossDelta) : 0
rating_after = max(0, rating_before + rating_delta)
```

Стартовый rating при первой записи:

```
rating_initial = max(0, start + rating_delta)
```

1.4.3. 4.3. Level curve

Level curve читается из meta_seasons.config.levels:

```
xpPerLevelSquaredBase = 100
level(xp) = max(1, floor(sqrt(xp / xpPerLevelSquaredBase)) + 1)
xp_floor(level) = xpPerLevelSquaredBase * (level - 1)^2
```

level	minimum XP
1	0
2	100
3	400
4	900
5	1600
10	8100

1.4.4. 4.4. Seasons

```
DefaultDurationDays = 14
DefaultPreparedSeasonCount = 30
```

Runtime:

1. истекший active season переводится в finished;
2. planned season, покрывающий now, переводится в active;
3. если planned нет, создается active season на 14 дней;
4. очередь planned seasons дозаполняется до 30 будущих сезонов.

Темы SeasonPlanFactory: balanced, quest_rush, high_roller, clan_wars, tournament_arc, jackpot_hunt, streak_sprint, risk_watch.

XP, rating start, rating delta и level curve применяются из JSON-конфига сезона.

Season rewards тоже читаются из JSON-конфига сезона:

```
rewards.playerTop = [5000, 2500, 1000]
rewards.clanTop   = [10000, 5000, 2500]
```

Фактическая выплата за место:

```
reward(place) =
    rewards[place - 1], если place в пределах массива
    0, иначе
```

Автоматический rollover закрывает истекший active season, активирует подходящий planned season и дозаполняет очередь будущих сезонов. Награды finished-сезонов обрабатываются идемпотентно через operation id, поэтому повтор job не должен удваивать выплаты.

1.5. 5. Quest rotation

Каталог квестов хранится в games/Games.Meta/quest-pool.json.

Активная ротация выбирается детерминированно:

seed = season_id : chat_id : user_id : period_key : slot_id : index

Периоды:

daily -> yyyy-MM-dd

weekly -> ISO yyyy-Www

Вес редкости:

rarity	weight
common	100
uncommon	60
rare	25
epic	10
legendary	4

meta_seasons.config.quests.focus сужает candidate pool, если в каталоге есть подходящие квесты:

all-round / balanced / normal -> без сужения
daily / weekly -> квесты указанного периода
volume -> volume/high_stake
payout -> payout/profit/multiplier
streaks -> streak tags/clusters
clans -> clan tags
tournaments -> tournament tags
controlled -> low_stake/play/loss

meta_seasons.config.quests.rarityBias меняет эффективный вес:

normal -> без изменения
uncommon -> uncommon * 2
rare -> rare/epic/legendary * 3
epic -> epic/legendary * 4
common -> common * 2, остальные / 2

Progress gate остается главным ограничением: редкие или тяжелые квесты не попадут новичку, если у них задан MinLevel, MinGamesPlayed или MinTotalStaked.

Progress gate:

quest unlocked iff
player.level >= MinLevel
and player.games_played >= MinGamesPlayed
and player.total_staked >= MinTotalStaked

Quest progress delta:

volume -> stake
payout -> payout
profit -> max(0, payout - stake)
other -> 1

1.6. 6. Faucets, sinks и drift

Основные sinks: отрицательное EV dice games, transfer fee, pick/pick lottery rake, horse pool hold, gas tax на slots.

Основные faucets: daily bonus, quest rewards, season rewards, clan rewards, redeem drops, Pick streak bonus.

Для периода T:

```
DeltaSupply(T) =  
  sum_game_payouts(T)  
  + sum_daily_bonus(T)  
  + sum_quest_coin_rewards(T)  
  + sum_season_rewards(T)  
  + sum_redeem_rewards(T)  
  - sum_stakes_debited(T)  
  - sum_transfer_fees(T)  
  - sum_lottery_fees(T)
```

Условие стабильной экономики:

```
daily_bonus + quest_coins + season_rewards + redeem_value  
  <= expected_house_edge + transfer_fees + lottery_fees
```

Админская страница /admin/meta-economy считает rough simulation для выбранных days, players, среднего stake/RTP и сезонных reward pool, а рядом показывает фактический ledger drift по economics_ledger. Это быстрый sanity check, не Monte Carlo.

1.7. 7. Быстрый баланс-свод

Модуль	RTP / эффект	Edge / sink
DiceCube	83.3333%	16.6667%
Darts	83.3333%	16.6667%
Bowling	83.3333%	16.6667%
Football	80.0000%	20.0000%
Basketball	80.0000%	20.0000%
Slots default	95.3125%	4.6875%
Pick base	около 95%	около 5%
Pick lottery	около 95%	около 5%
Horse	зависит от pool	(P - S_winner) / 11
Transfer	нет RTP	комиссия 3%, минимум 1
Daily bonus	faucet	до 8 монет/день